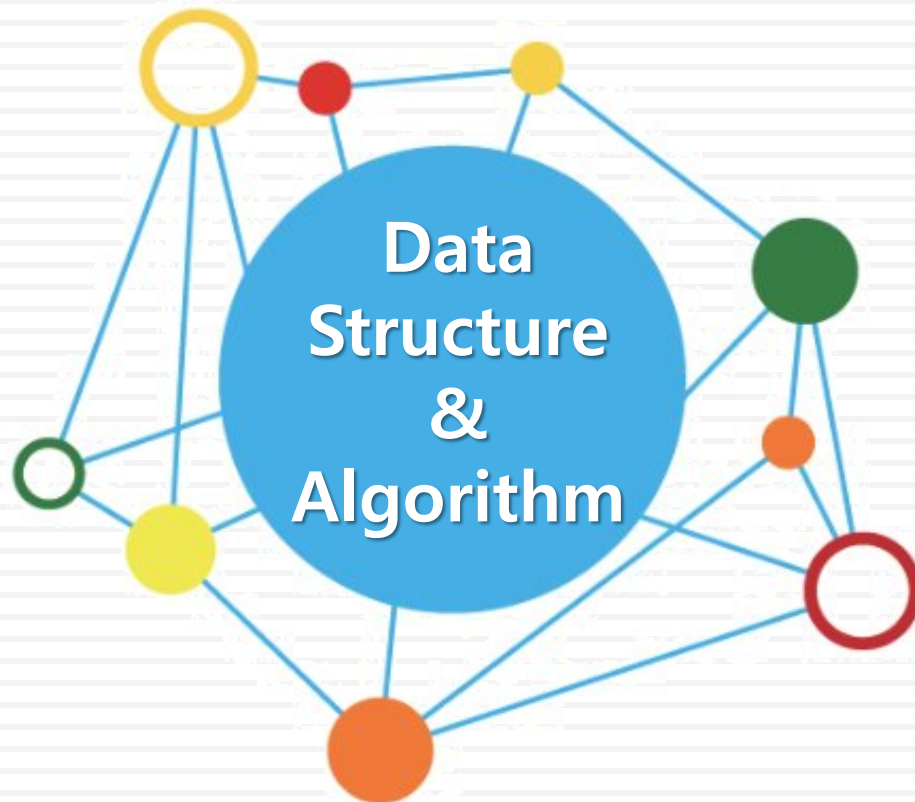


데이터 구조론



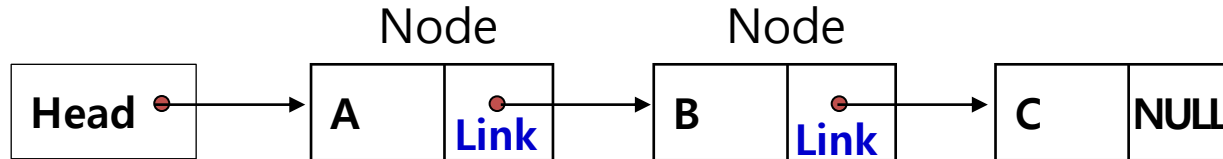
유길현

연결 데이터구조와
연결 리스트

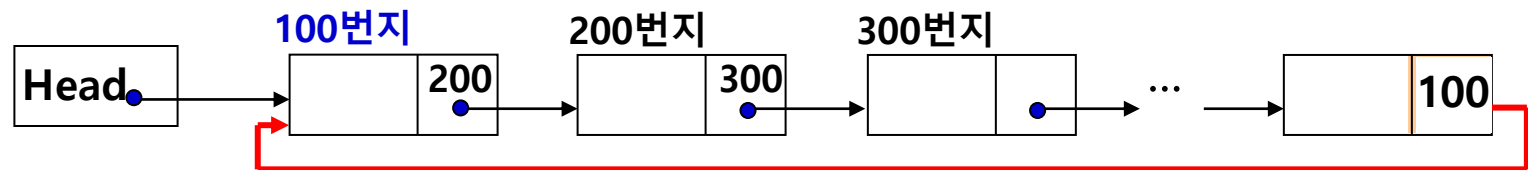
연결 리스트의 종류

❖ 연결 리스트

■ 단순 연결 리스트



■ 원형 연결 리스트

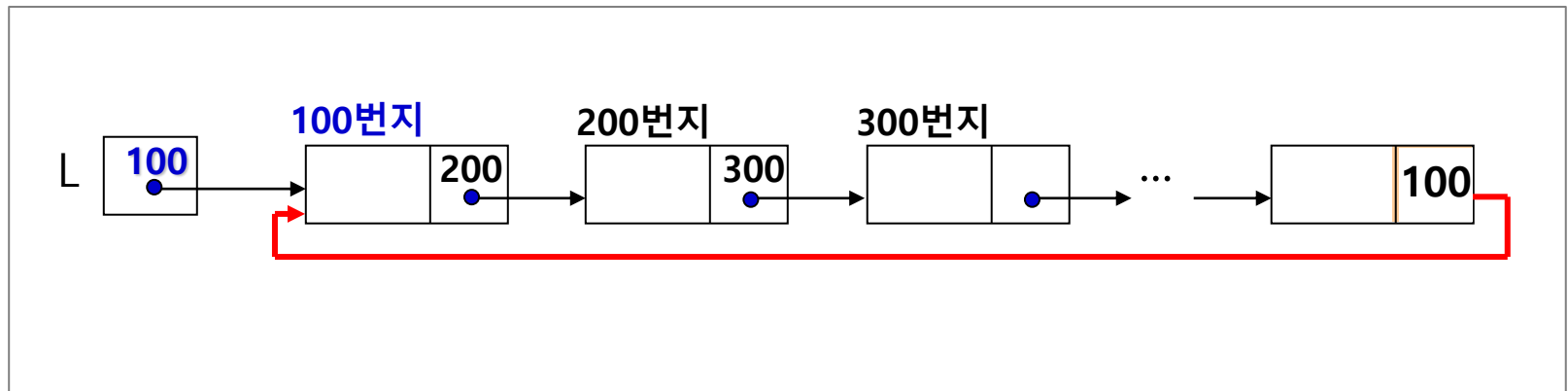


■ 이중 연결 리스트



❖ 원형 연결 리스트(Circular Linked List)의 정의

- 단순 연결 리스트에서 마지막 노드가 리스트의 첫 번째 노드를 가리키게 하여 리스트의 구조를 원형으로 만든 연결 리스트
- 단순 연결 리스트의 마지막 노드의 링크 필드에 첫 번째 노드의 주소를 저장하여 구성
- 링크를 따라 계속 순회하면 이전 노드에 접근 가능
- 원형 리스트에서도 단순 연결 리스트와 마찬가지로 시작 위치를 알려주는 헤더 포인터가 있고 다른 노드와 원형을 이루고 있다.



❖ 원형 연결 첫 번째 노드 삽입 알고리즘

- 마지막 노드의 링크를 첫 번째 노드로 연결하는 부분만 제외하고는 단순 연결 리스트에서의 삽입 연산과 같은 연산

알고리즘

원형 연결 리스트의 첫 번째 노드 삽입

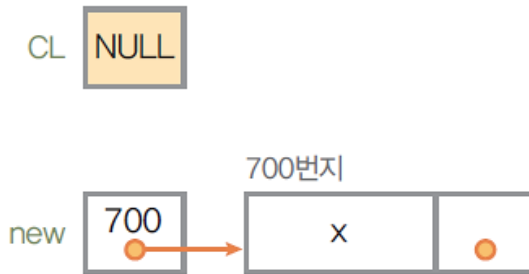
```
insertFirstNode(CL, x)
  new ← getNode( );
  new.data ← x;
  if(CL == NULL) then {
    ①-① CL ← new;
    ①-② new.link ← new;
  }
  else {
    ②-① temp ← CL;
    while(temp.link ≠ CL) do
      ②-② temp ← temp.link;
    ②-③ new.link ← temp.link;
    ②-④ temp.link ← new;
    ②-⑤ CL ← new;
  }
end insertFirstNode;
```

❖ 원형 연결 첫 번째 노드로 삽입하는 알고리즘

- 원형 연결 리스트 첫번째에 노드를 삽입하는 과정

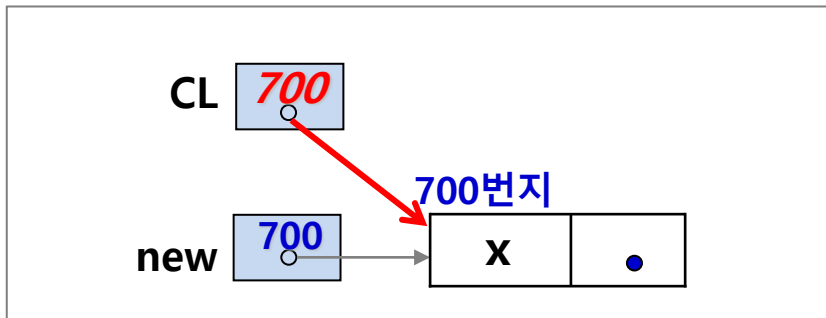
① 공백 리스트인 경우

삽입하는 노드 new는 리스트 첫 번째 노드이자 마지막 노드가 되어야 한다.



①-① CL ← new;

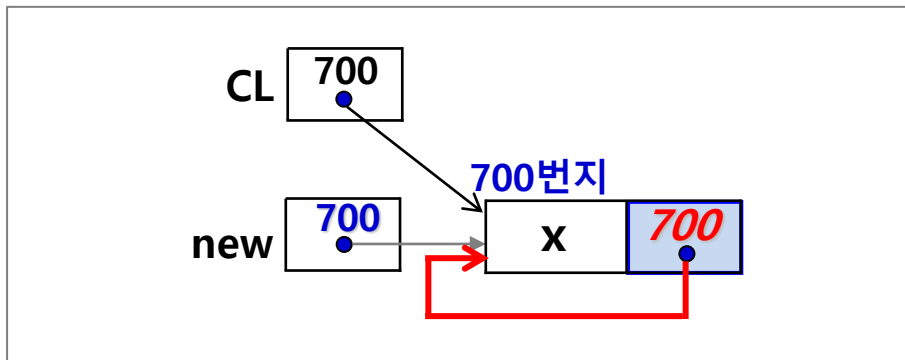
포인터 CL이 노드 new를 가리키게 한다.



❖ 원형 연결 첫 번째 노드로 삽입하는 알고리즘

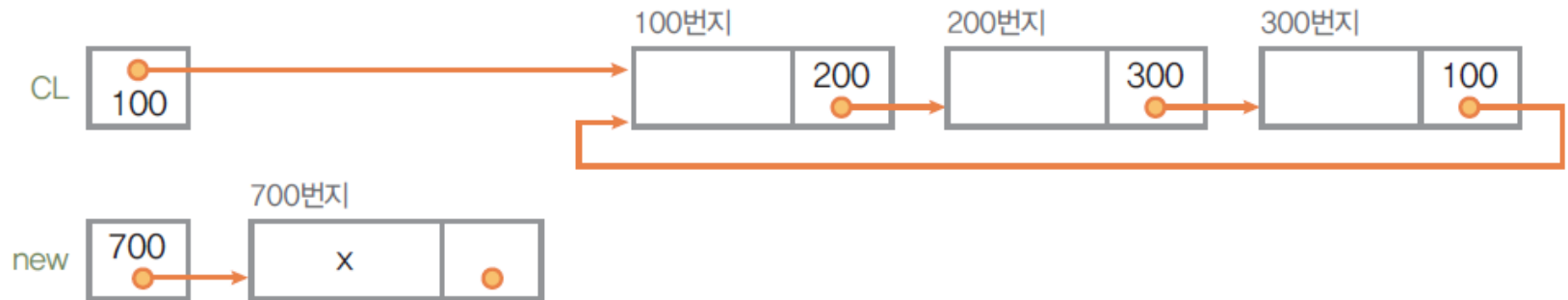
①-⑥ new.link ← new;

노드 new가 자기자신을 가리키게 함으로써 노드 new를 첫 번째 노드이자 마지막 노드가 되도록 한다.



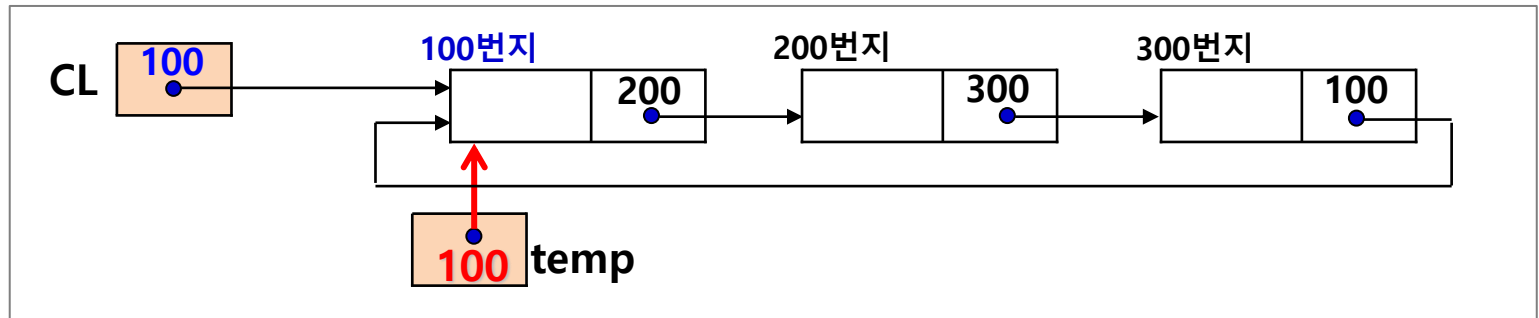
❖ 원형 연결 첫 번째 노드로 삽입하는 알고리즘

② 공백 리스트가 아닌 경우



②-① temp ← CL;

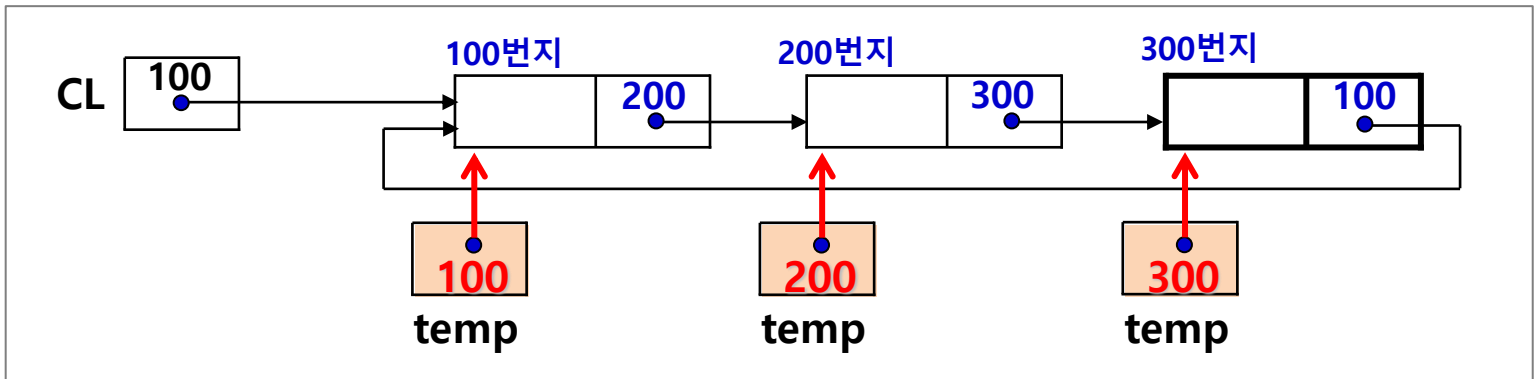
리스트가 공백리스트가 아닌 경우에는 첫 번째 노드의 주소를 임시 순회 포인터 temp에 저장하여 노드 순회의 시작점을 지정한다.



❖ 원형 연결 첫 번째 노드로 삽입하는 알고리즘

②-⑥ temp ← temp.link;

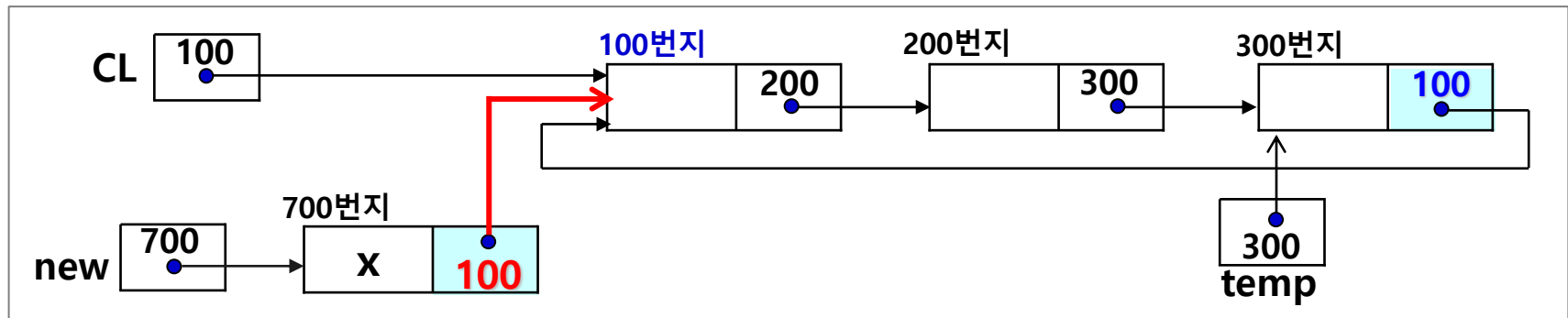
while 문을 수행해 순회 포인터 temp를 링크를 따라 마지막 노드 까지 이동



❖ 원형 연결 첫 번째 노드로 삽입하는 알고리즘

②-③ new.link ← temp.link;

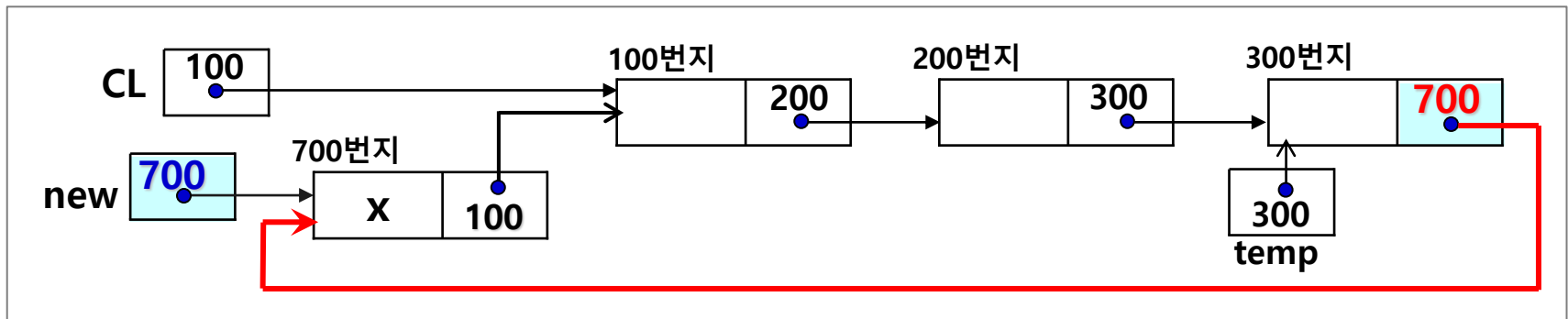
리스트의 마지막 노드의 링크 값을 노드 new의 링크에 저장한다.
노드 new는 노드 temp의 다음 노드인 첫 번째 노드와 연결된다.



❖ 원형 연결 첫 번째 노드로 삽입하는 알고리즘

②-④ temp.link ← new;

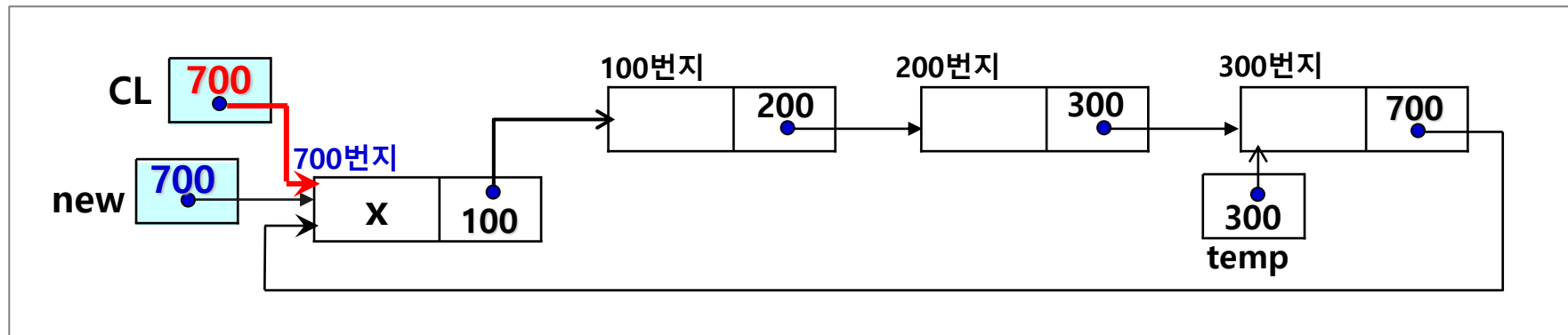
포인터 new의 값을 포인터 temp가 가리키고 있는 마지막 노드의 링크에 저장하여, 리스트의 마지막 노드가 노드 new를 가리키게 한다.



❖ 원형 연결 첫 번째 노드로 삽입하는 알고리즘

②-⑤ CL ← new;

노드 new의 값을 리스트 포인터 CL에 저장하여 노드 new가 리스트의 첫 번째 노드가 되도록 지정



- CL이 공백이 아닌 경우 첫번째 노드로 삽입한 최종 결과



❖ 원형 연결 리스트 첫 번째 노드 삽입 연산 코드

```
void insertFirstNode(linkedList_h *CL, char *x)
{
    listNode* newNode, *temp;
    // 삽입할 새 노드 할당
    newNode = (listNode*)malloc(sizeof(listNode));
    strcpy(newNode->data, x);
    if (CL->head == NULL) {    // 원형 연결 리스트가 공백인 경우
        CL->head = newNode;    // 새 노드를 리스트의 시작 노드로 연결
        newNode->link = newNode;
    }
    else {                    // 원형 연결 리스트가 공백이 아닌 경우
        temp = CL->head;
        while (temp->link != CL->head)
            temp = temp->link;
        newNode->link = temp->link;
        // 마지막 노드를 첫 번째 노드인 new와 원형 연결
        temp->link = newNode;
        CL->head = newNode;
    }
}
```

❖ 원형 연결 리스트 중간에 노드를 삽입하는 알고리즘

알고리즘

원형 연결 리스트의 중간 노드 삽입

```
insertMiddleNode(L, pre, x)
    new ← getNode( );
    new.data ← x;
    ① { if (CL = NULL) then {
        CL ← new;
        new.link ← new;
    }
    ② { else{
        ②-㉠ new.link ← pre.link;
        ②-㉡ pre.link ← new;
    }
    end insertMiddleNode()
```

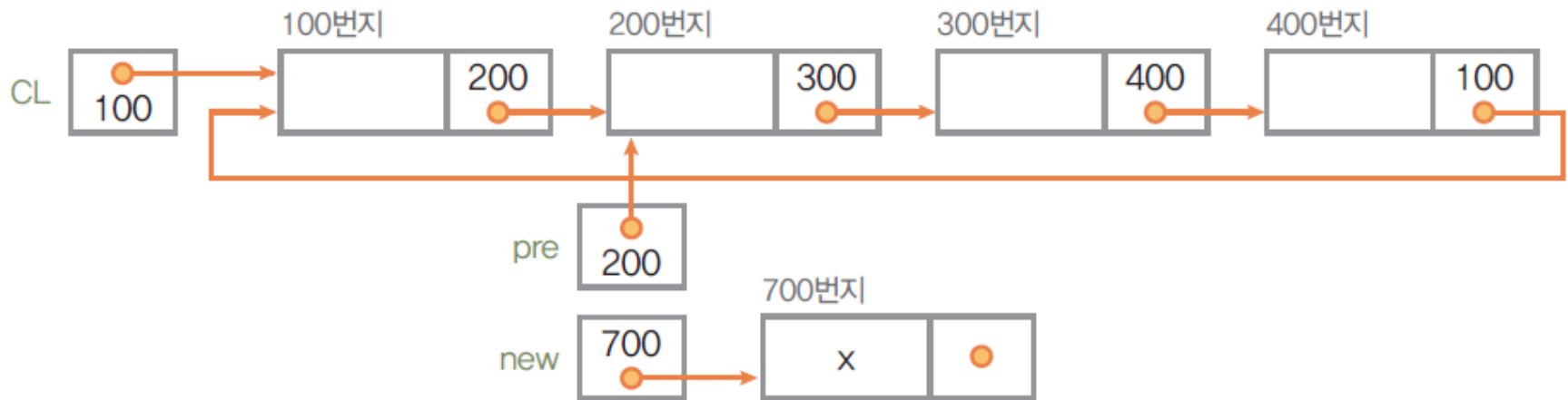
❖ 원형 연결 리스트 중간에 노드를 삽입하는 과정

① 공백 리스트인 경우

CL NULL



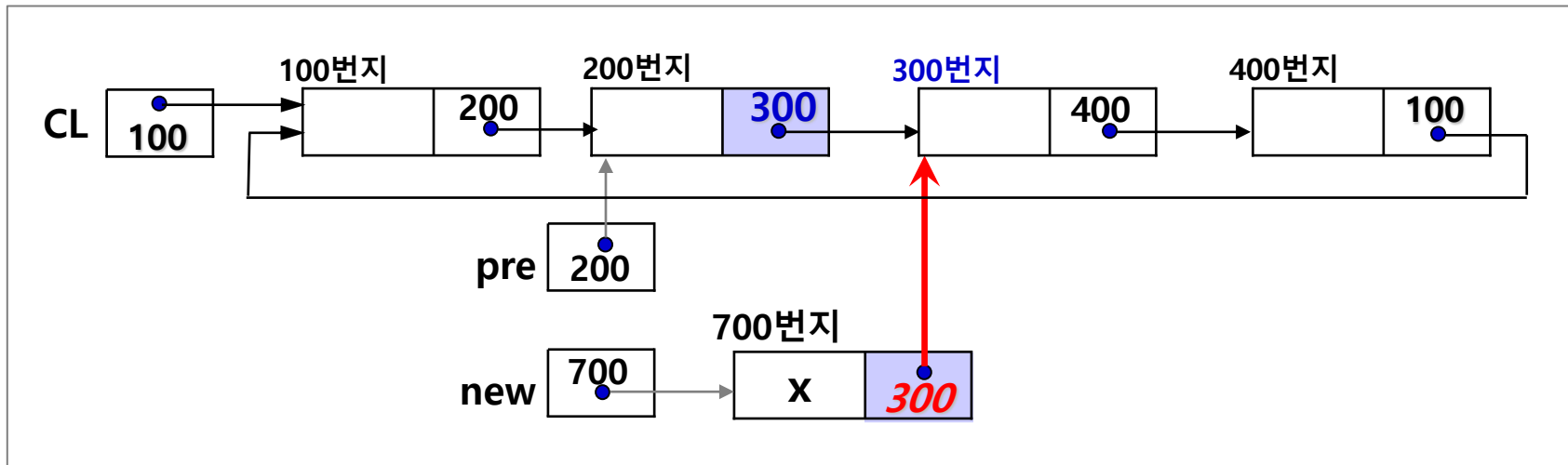
② 공백 리스트가 아닌 경우



❖ 원형 연결 리스트 중간에 노드를 삽입하는 과정

②-① new.link ← pre.link

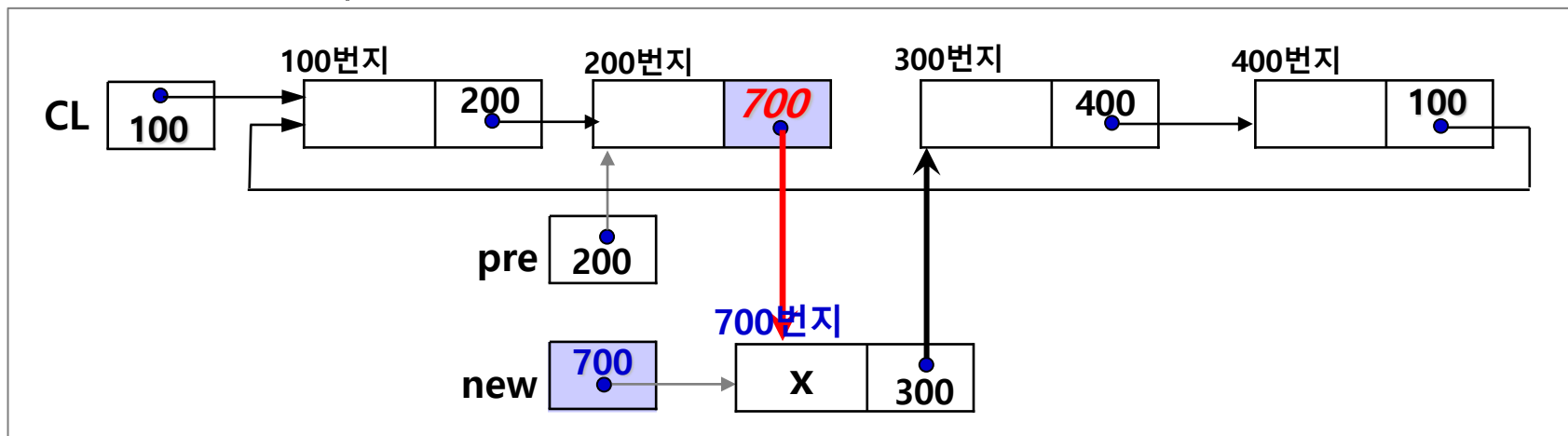
노드 pre의 다음 노드로 new를 삽입하기 위해서, 먼저 **노드 pre**의 다음 노드(pre.link)를 **new의 다음 노드(new.link)**로 연결



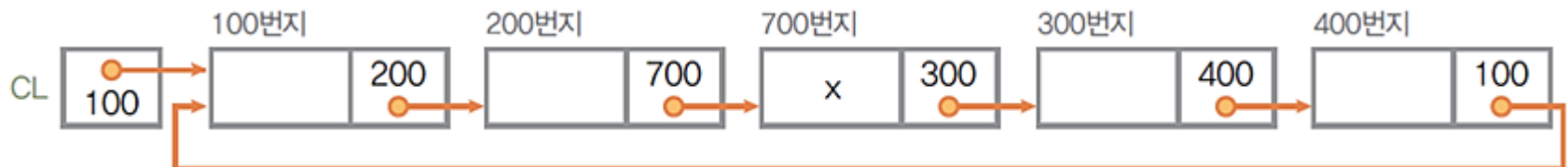
❖ 원형 연결 리스트 중간에 노드를 삽입하는 과정

②-⑥ pre.link ← new

노드 new의 값(삽입할 노드의 주소)을 노드 pre의 링크에 저장하여, 노드 pre가 노드 new를 가리키도록 한다.



- 원형 연결 리스트 CL인 공백이 아닌 경우에 중간 노드로 삽입한 최종 결과



❖ 원형 연결 리스트 중간에 노드 삽입 연산 코드

```
void insertMiddleNode(linkedList_h* CL, listNode *pre, char*x)
{
    listNode* newNode;
    newNode = (listNode*)malloc(sizeof(listNode));
    strcpy(newNode->data, x);
    if (CL == NULL)
    {
        CL->head = newNode;
        newNode->link = newNode;
    }
    else
    {
        newNode->link = pre->link;
        pre->link = newNode;
    }
}
```

❖ 노드 삭제 알고리즘

알고리즘

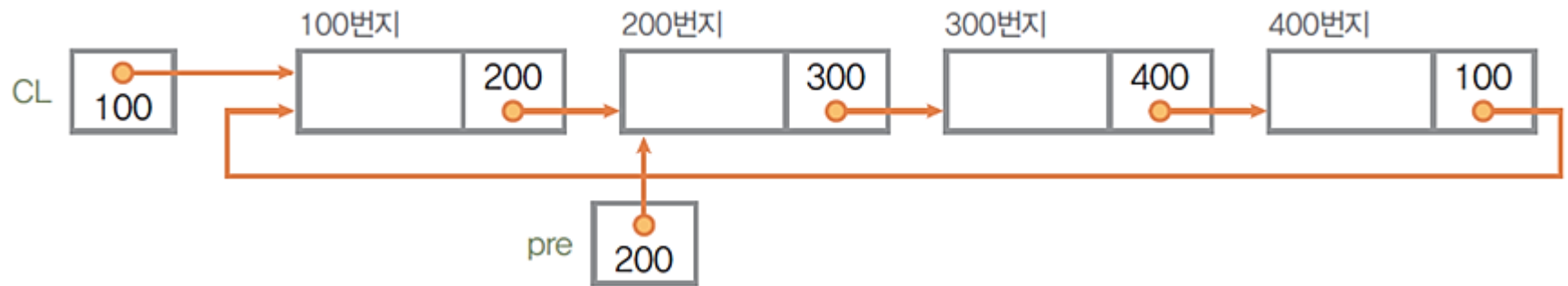
원형 연결 리스트의 노드 삭제

```
deleteNode(CL, pre)
  if(CL == NULL) then error;
  else {
    ① old ← pre.link;
    ② pre.link ← old.link;
    ③ { if (old == CL) then
        ③-a CL ← old.link;
    ④ returnNode(old);
  }
end deleteNode()
```

원형 연결 리스트 노드 삭제

❖ 원형 연결 리스트에서 노드를 삭제하는 과정

■ 초기 상태

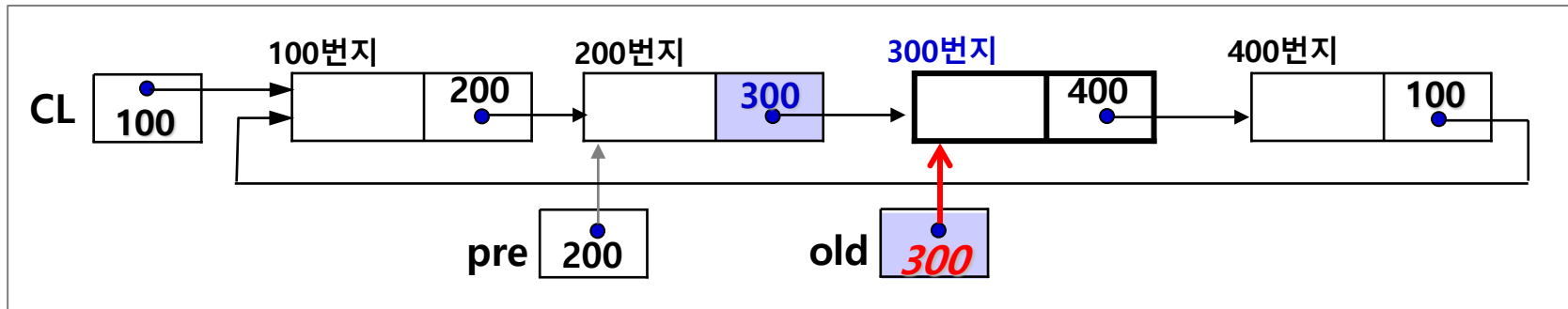


원형 연결 리스트 노드 삭제

❖ 원형 연결 리스트에서 노드를 삭제하는 과정

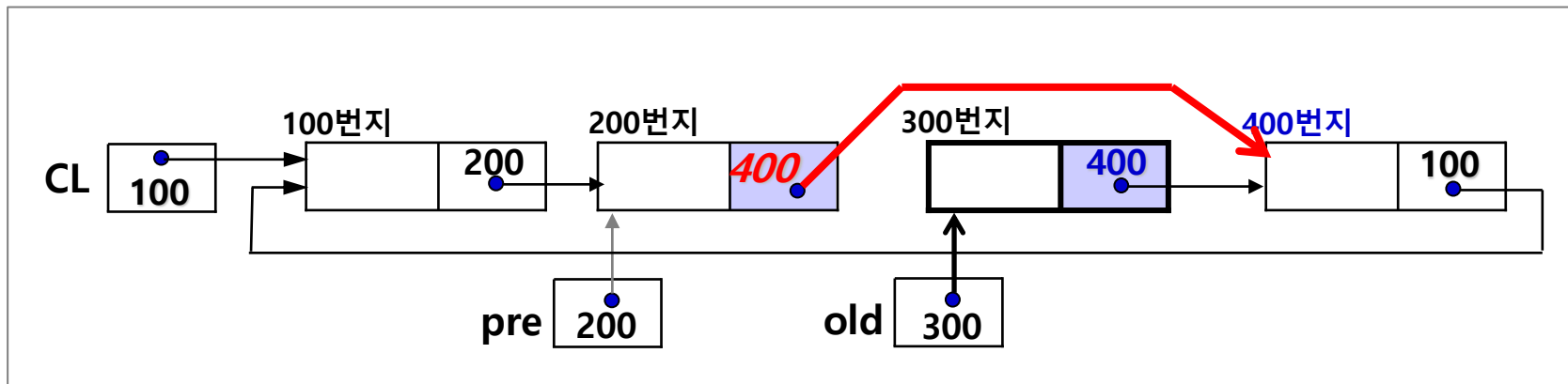
① $old \leftarrow pre.link;$

노드 pre 의 다음노드($pre.link$)를 삭제할 노드 old 로 지정



② $pre.link \leftarrow old.link;$

old 의 다음노드($old.link$)를 노드 old 의 이전노드 pre 노드와 서로 연결

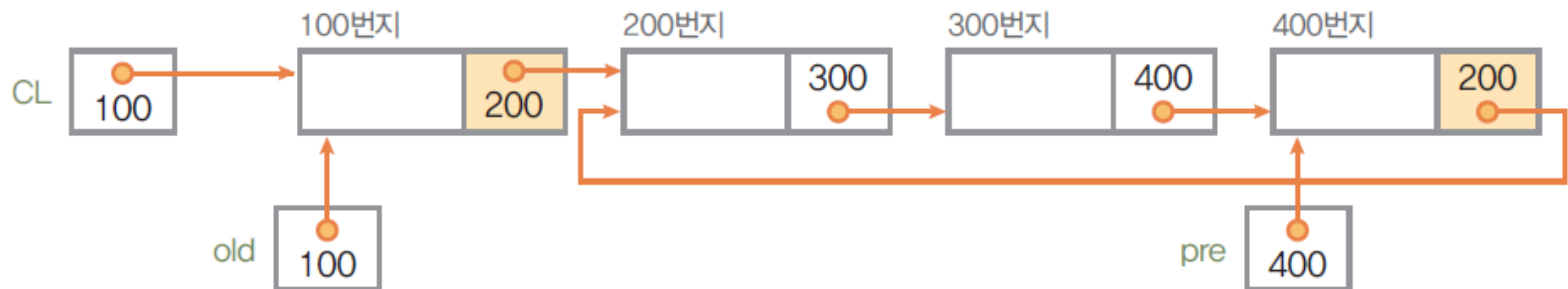


원형 연결 리스트 노드 삭제

❖ 원형 연결 리스트에서 노드를 삭제하는 과정

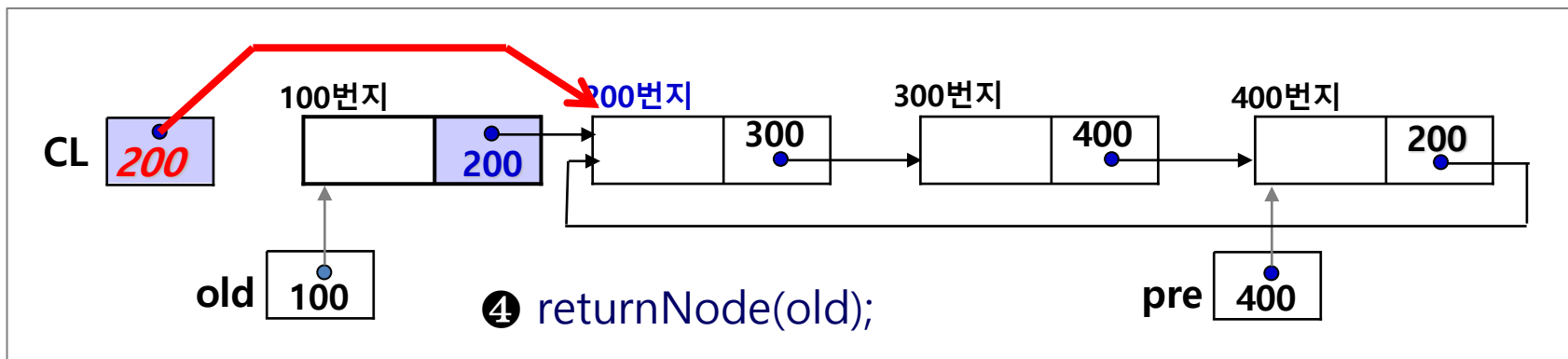
③ if(old == CL) then

삭제할 노드 old가 원형 연결 리스트의 첫번째 노드인 경우 >>



③-① CL ← old.link;

첫 번째 노드를 삭제하는 경우에는 **노드 old의 링크 값을 리스트 포인터 CL에** 저장하여 두 번째 노드가 리스트의 첫 번째 노드가 되도록 조정

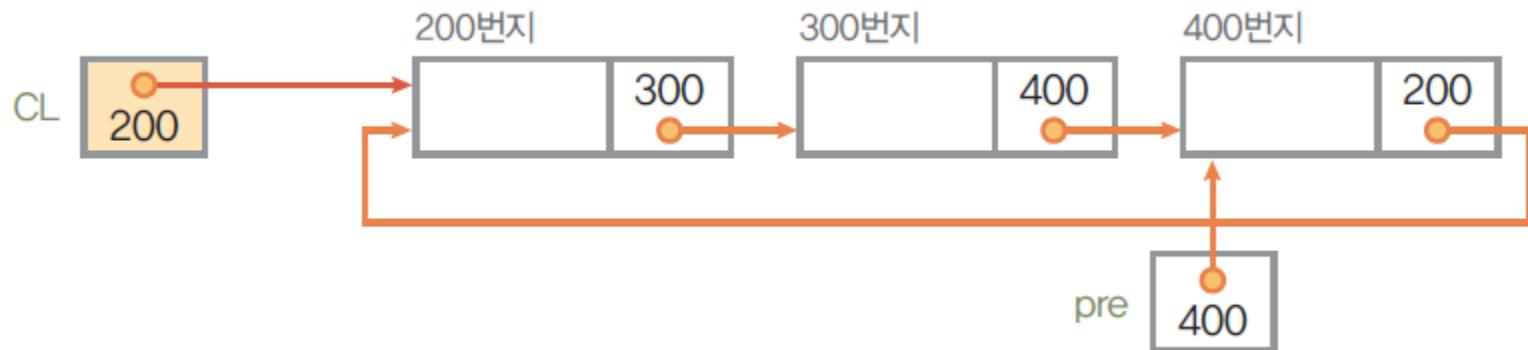


❖ 원형 연결 리스트에서 노드를 삭제하는 과정

④ returnNode(old);

삭제한 노드 old의 메모리를 반환한다.

■ 최종 결과



❖ 원형 연결 리스트 노드 삭제 코드(1/2)

```
void deleteNode(linkedList_h* CL, listNode* old)
{
    // 삭제할 노드의 선행자 노드를 나타내는 포인터
    listNode* pre;
    if (CL->head == NULL)
        return;          // 공백 리스트인 경우, 삭제 연산 중단
    // 리스트에 노드가 한 개만 있는 경우
    if (CL->head->link == CL->head)
    {
        free(CL->head);    // 첫 번째 노드의 메모리를 해제하고
        CL->head = NULL;    // 리스트 시작 포인터를 NULL로 설정
        return;
    }
}
```

❖ 원형 연결 리스트 노드 삭제 코드(2/2)

```
else if (old == NULL)
    return;          // 삭제할 노드가 없는 경우, 삭제 연산 중단
else
{
    pre = CL->head; // 포인터 pre를 리스트의 시작 노드에 연결
    while (pre->link != old)
    {
        // 선행자 노드를 포인터 pre를 이용해 찾음
        pre = pre->link;
    }
    pre->link = old->link;
    if (old == CL->head)
        CL->head = old->link;
    free(old);      // 삭제 노드의 메모리를 해제
}
}
```


❖ 원형 연결 리스트의 노드 구조를 구조체로 선언

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#include "string.h"

// 원형 연결 리스트의 노드 구조를 구조체로 정의
typedef struct ListNode {
    char data[4];
    struct ListNode* link;
} ListNode;

// 리스트의 시작을 나타내는 head 노드를 구조체로 정의
typedef struct {
    ListNode* head;
} linkedList_h;
```

❖ 공백 원형 연결 리스트를 생성

```
// 공백 원형 연결 리스트를 생성하는 연산
linkedList_h* createLinkedList_h(void)
{
    linkedList_h* CL;
    // 헤드 노드 할당
    CL = (linkedList_h*)malloc(sizeof(linkedList_h));
    CL->head = NULL;    // 공백 리스트이므로 NULL로 설정
    return CL;
}
```

❖ 원형 연결 리스트를 순서대로 출력

```
// 연결 리스트를 순서대로 출력하는 연산
void printList(linkedList_h* CL)
{
    listNode* p;
    printf(" CL = (");
    p = CL->head;
    do {
        printf("%s", p->data);
        p = p->link;
        if (p != CL->head)
            printf(", ");
    } while (p != CL->head);
    printf(") \n");
}
```

❖ 원형 연결 리스트에서 노드를 탐색

```
listNode* searchNode(linkedList_h* CL, char* x)
{
    listNode *temp;
    temp = CL->head;
    if (temp == NULL) return NULL;
    do {
        if (strcmp(temp->data, x) == 0)
            return temp;
        else
            temp = temp->link;
    } while (temp != CL->head);
    return NULL;
}
```

❖ 프로그램을 실행시키기 위한 메인 함수 만들기

```
int main()
{
    linkedList_h* CL;
    listNode *p;
    CL = createLinkedList_h();    // 공백 원형 연결 리스트 생성
    printf("(1) 원형 연결 리스트 생성하기! \n");
    getchar();                    // 현재까지의 출력 화면 확인

    printf("(2) 원형 연결 리스트에 [월] 노드 삽입하기! \n");
    insertFirstNode(CL, "월");
    printList(CL);
    getchar();                    // 현재까지의 출력 화면 확인

    printf("(3) 원형 연결 리스트의 [월] 노드 뒤에 [수] 노드 삽입하기! \n");
    p = searchNode(CL, "월");
    insertMiddleNode(CL, p, "수");
    printList(CL); getchar();    // 현재까지의 출력 화면 확인
}
```

❖ 프로그램을 실행시키기 위한 메인 함수 만들기

```
printf("(4) 원형 연결 리스트의 [수] 노드 뒤에 [금] 노드 삽입하기! \n");
p = searchNode(CL, "수");
insertMiddleNode(CL, p, "금");
printList(CL);
getchar();                                // 현재까지의 출력 화면 확인

printf("(5) 원형 연결 리스트에서 [수] 노드 삭제하기! \n");
p = searchNode(CL, "수");
deleteNode(CL, p);
printList(CL);
getchar();                                // 현재까지의 출력 화면 확인

return 0;
}
```

❖ 원형 연결 리스트 프로그램 : 교재 190p

■ 실행 결과

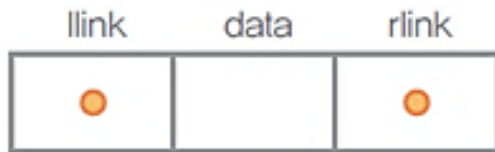
```
C:\한국산업기술대학교\2021년1학기\메카트로닉스\실습예제\원형연결리스...
(1) 원형 연결 리스트 생성하기!
(2) 원형 연결 리스트에 [월] 노드 삽입하기!
CL = (월)
(3) 원형 연결 리스트의 [월] 노드 뒤에 [수] 노드 삽입하기!
CL = (월, 수)
(4) 원형 연결 리스트의 [수] 노드 뒤에 [금] 노드 삽입하기!
CL = (월, 수, 금)
(5) 원형 연결 리스트에서 [수] 노드 삭제하기!
CL = (월, 금)

-----
Process exited after 17.62 seconds with return value 0
계속하려면 아무 키나 누르십시오 . . .
```

❖ 이중 연결 리스트의 개념

- 양쪽 방향으로 순회할 수 있도록 노드를 연결한 리스트
- 이중 연결 리스트의 노드 구조와 구조체 정의
 - llink(left link) 필드 : 왼쪽노드와 연결하는 포인터
 - rlink(right link) 필드 : 오른쪽 노드와 연결하는 포인터

노드 구조



구조체 정의

```
typedef struct Dnode {  
    struct Dnode *llink;  
    char data[5];  
    struct Dnode *rlink;  
}
```


❖ 이중 연결 리스트 개념



(a) 단방향 기차놀이



(b) 양방향 기차놀이

- (b)의 양방향 기차를 이중 연결 리스트라고 생각하면 아이들은 노드
- 왼손의 이름표는 llink, 오른손의 이름표는 rlink

❖ 이중 연결 리스트에서 노드를 삽입하는 방법

- ❶ 삽입할 노드를 준비한다.
- ❷ 새노드의 데이터 필드 값을 저장한다.
- ❸ 새노드 왼쪽노드의 오른쪽 링크 필드(rlink)이 있던 값을 새노드의 오른쪽 링크 필드(rlink)에 저장한다.
- ❹ 왼쪽 노드의 오른쪽 링크 필드(llink)에 새 노드의 주소를 저장한다.
- ❺ 새 노드 오른쪽 노드의 왼쪽 링크 필드(llink)에 있던 값을 새 노드의 왼쪽 링크 필드(llink)에 저장한다.
- ❻ 오른쪽 노드의 왼쪽 링크 필드(llink)에 새 노드의 주소를 저장한다.
- ❼ 노드를 순서대로 연결한다.

❖ 이중 연결 리스트에 노드 삽입 알고리즘

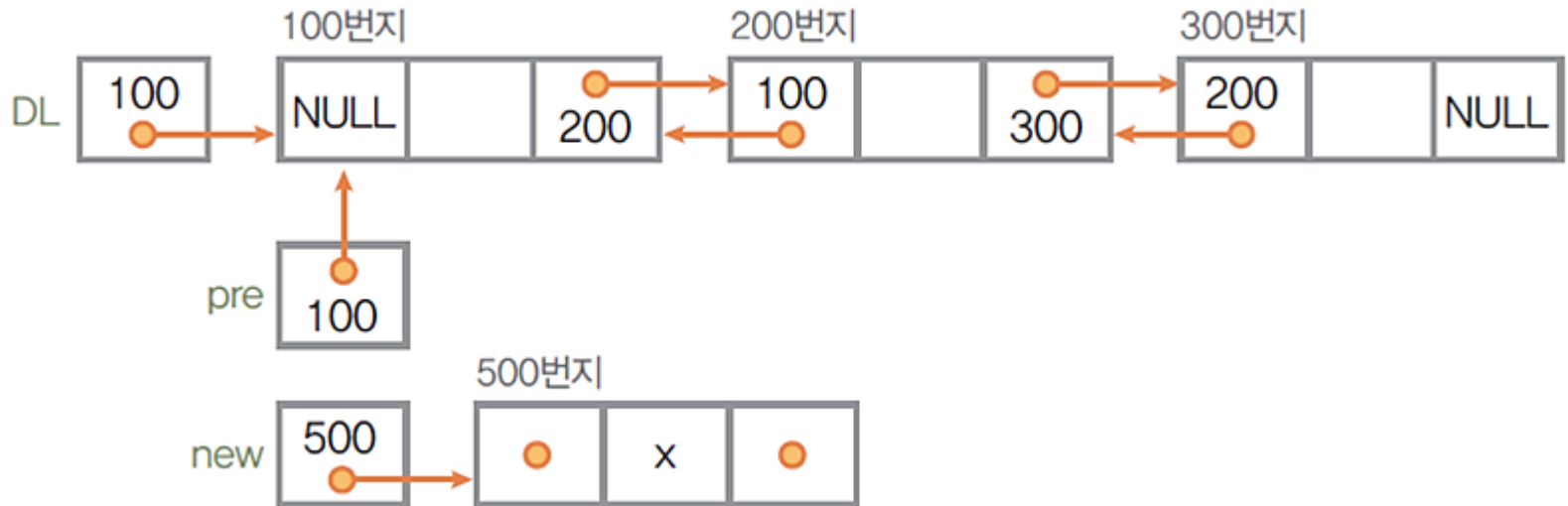
알고리즘

이중 연결 리스트의 노드 삽입

```
insertNode(DL, pre, x)
    new ← getNode( );
    new.data ← x;
    ① new.rlink ← pre.rlink;
    ② pre.rlink ← new;
    ③ new.llink ← pre;
    ④ new.rlink.llink ← new;
end insertNode()
```

❖ 이중 연결 리스트에 노드를 삽입하는 과정

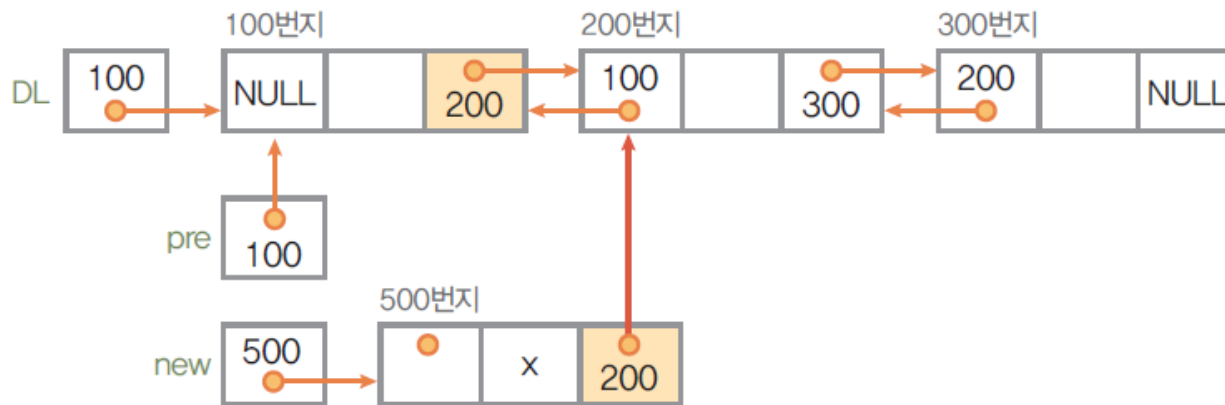
- 데이터 필드 값이 x인 노드 new를 준비, 포인터 pre가 가리키는 노드의 다음 노드로 삽입하려고 한다고 가정
 - 초기 상태



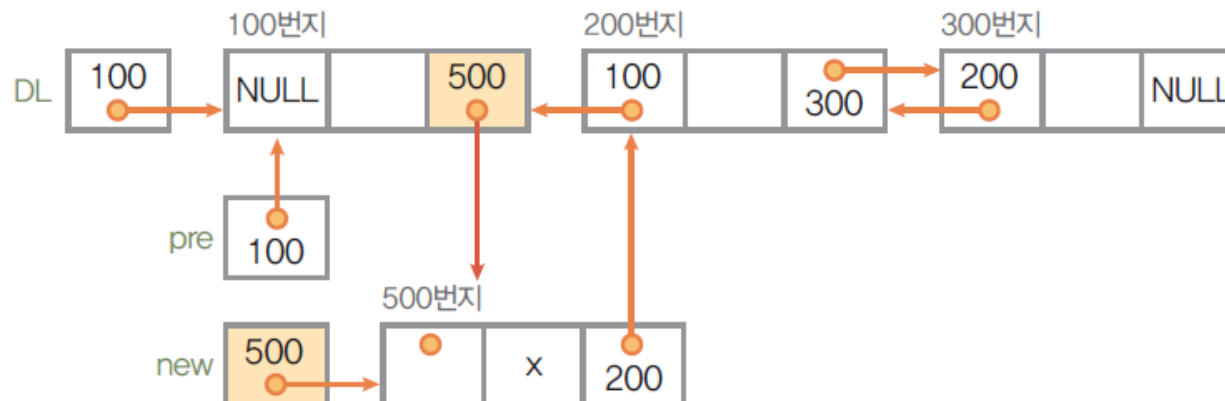
이중 연결 리스트 노드 삽입

❖ 이중 연결 리스트에 노드를 삽입하는 과정

① $\text{new.rlink} \leftarrow \text{pre.rlink};$



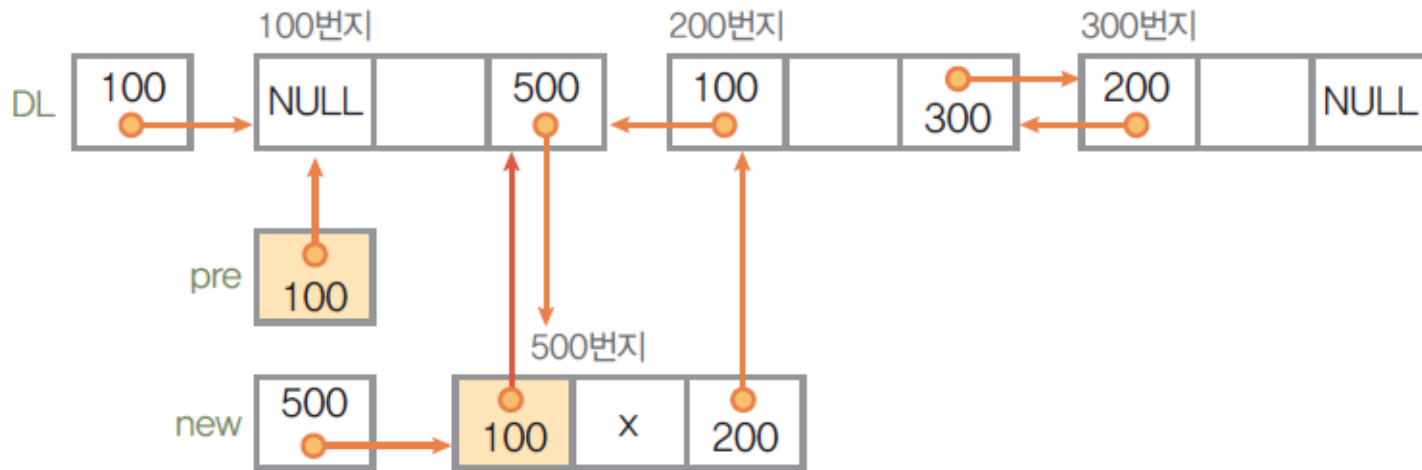
② $\text{pre.rlink} \leftarrow \text{new};$



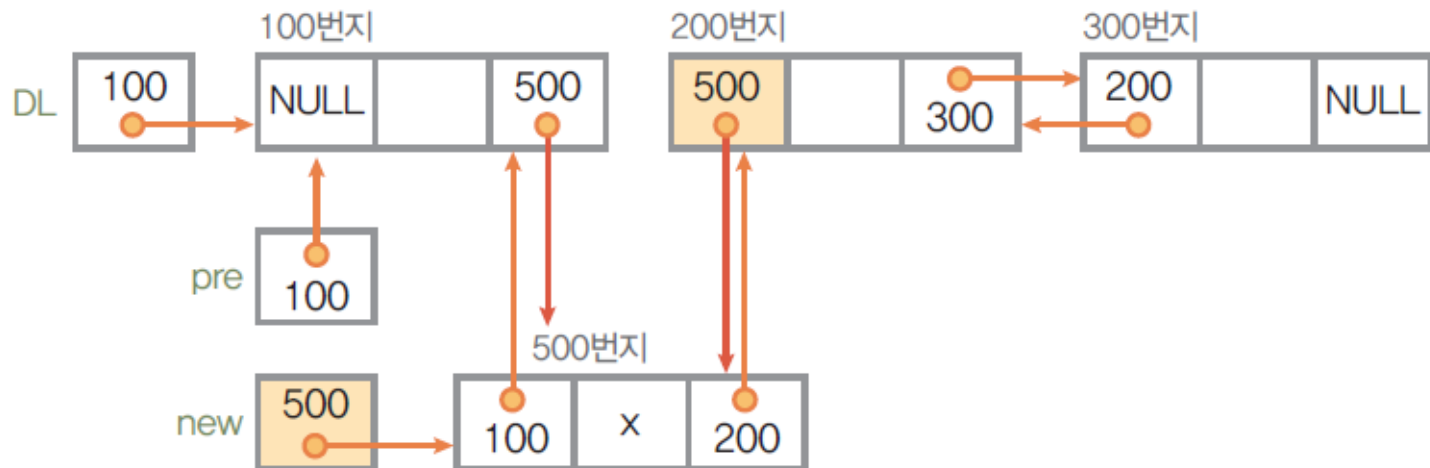
이중 연결 리스트 노드 삽입

❖ 이중 연결 리스트에 노드를 삽입하는 과정

③ $\text{new.link} \leftarrow \text{pre};$



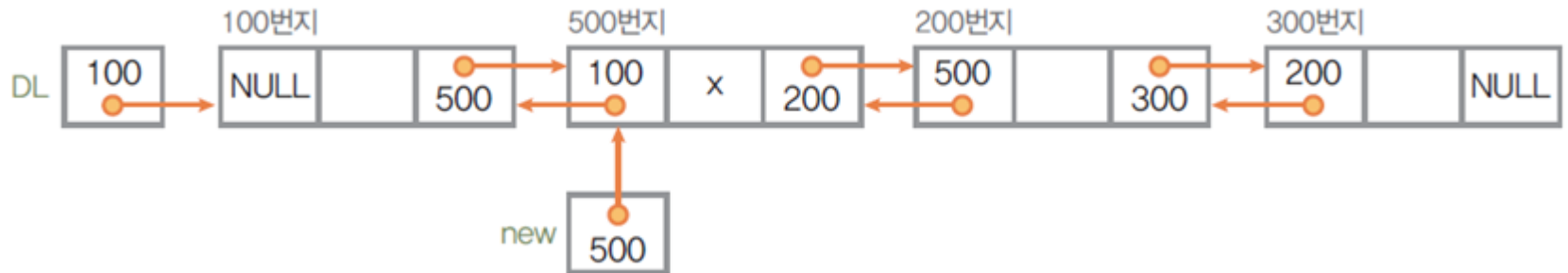
④ $\text{new.rlink.link} \leftarrow \text{new};$



이중 연결 리스트 노드 삽입

❖ 이중 연결 리스트에 노드를 삽입

- DL에 new 노드가 삽입된 최종 결과



❖ 이중 연결 리스트 노드 삽입 연산 코드

```
void insertNode(linkedList_h* DL, listNode *pre, char*x)
{
    listNode* newNode;
    newNode = (listNode*)malloc(sizeof(listNode));
    strcpy(newNode->data, x);
    if (DL->head == NULL) {
        newNode->rlink = NULL;
        newNode->llink = NULL;
        DL->head = newNode;
    }
    else {
        newNode->rlink = pre->rlink;
        pre->rlink = newNode;
        newNode->llink = pre;
        if (newNode->rlink != NULL) // 삽입 자리에 다음 노드가 있는 경우
            newNode->rlink->llink = newNode;
    }
}
```


❖ 이중 연결리스트에 노드를 삭제

■ 이중 연결 리스트에서 노드를 삭제하는 방법

- ❶ 삭제할 노드의 오른쪽 노드와 왼쪽 노드를 찾는다.
- ❷ 삭제할 노드의 오른쪽 노드의 주소(old.rlink)를 삭제할 노드의 왼쪽 노드(old.llink)의 오른쪽 링크 필드(rlink)에 저장한다.
- ❸ 삭제할 노드의 왼쪽 노드(old.llink)의 주소를 삭제할 노드의 오른쪽 노드(old.rlink)의 왼쪽 링크 필드에 저장한다.
- ❹ 노드를 순서대로 연결한다.

❖ 이중 연결리스트에 노드를 삭제

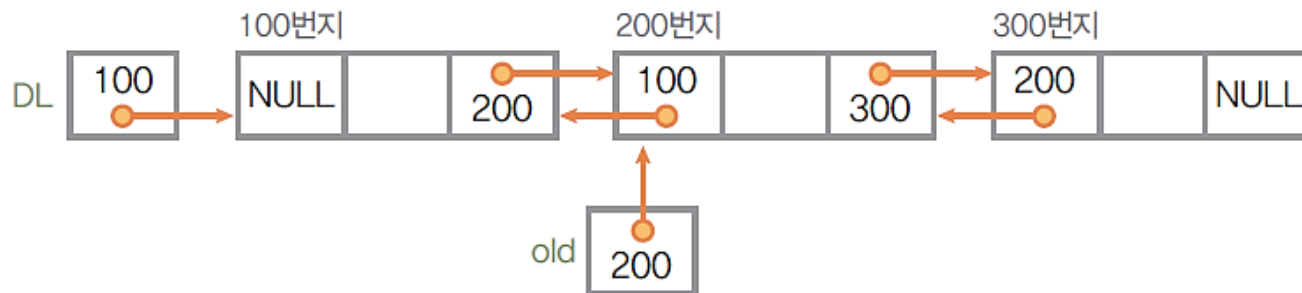
■ 알고리즘의 정의

알고리즘	이중 연결 리스트의 노드 삭제
<pre>deleteNode(DL, old) ① old.llink.rlink ← old.rlink; ② old.rlink.llink ← old.llink; ③ returnNode(old); end deleteNode()</pre>	

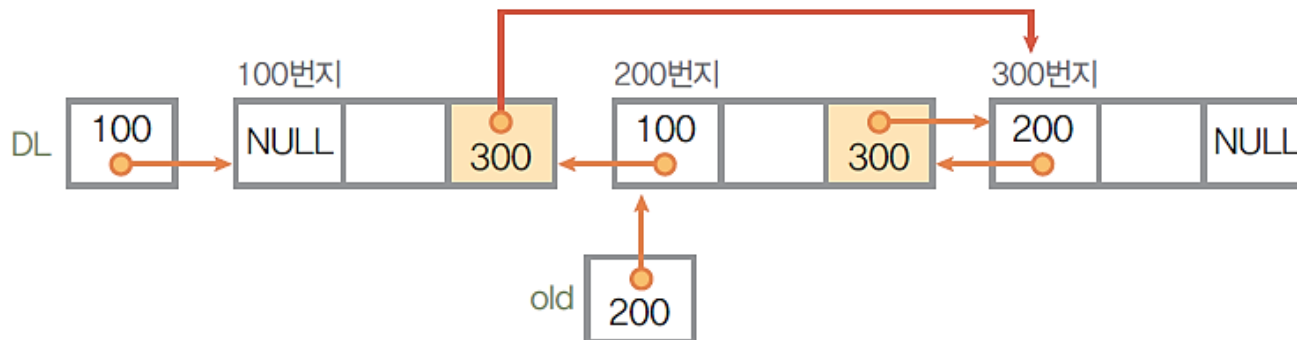
이중 연결 리스트 노드 삭제

❖ 이중 연결 리스트에서 노드를 삭제하는 과정

■ 초기상태



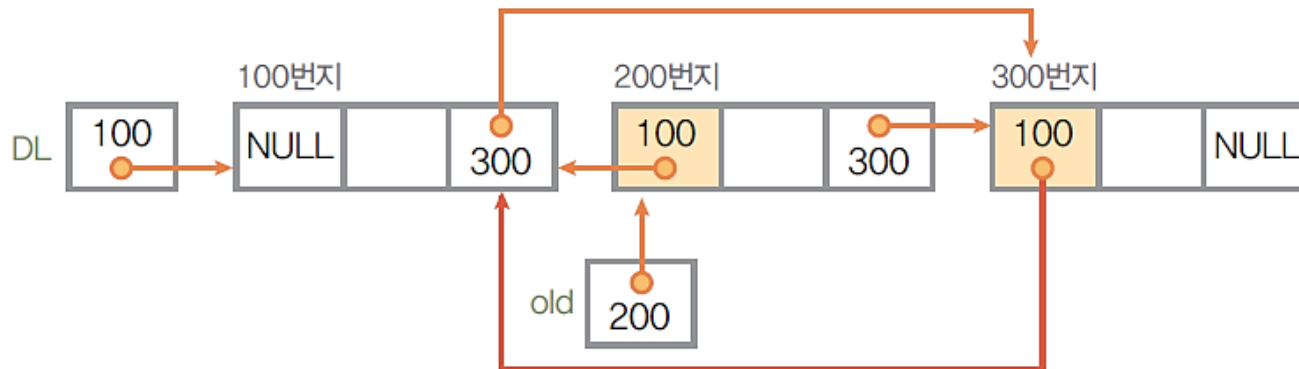
① `old.link.rlink ← old.rlink;`



이중 연결 리스트 노드 삭제

❖ 이중 연결 리스트에서 노드를 삭제하는 과정

② $\text{old.rlink.link} \leftarrow \text{old.llink}$;



③ $\text{returnNode}(\text{old})$;

■ 최종 결과



❖ 이중 연결리스트 노드 삭제연산 코드

```
void deleteNode(linkedList_h* DL, listNode* old)
{
    if (DL->head == NULL) // 공백 리스트인 경우, 삭제 연산 중단
        return;
    else if (old == NULL) // 삭제할 노드가 없는 경우, 삭제 연산 중단
        return;
    else {
        old->llink->rlink = old->rlink;
        old->rlink->llink = old->llink;
        free(old);          // 삭제 노드의 메모리 해제
    }
}
```

❖ 이중 연결 리스트의 노드 구조를 구조체로 선언

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
#include "stdlib.h"
#include "string.h"

// 이중 연결 리스트의 노드 구조를 구조체로 정의
typedef struct ListNode {
    struct ListNode* llink;    // 왼쪽(선행) 노드에 대한 링크
    char data[4];
    struct ListNode* rlink;    // 오른쪽(다음) 노드에 대한 링크
} ListNode;

// 리스트 시작을 나타내는 head 노드를 구조체로 정의
typedef struct {
    ListNode* head;
} linkedList_h;
```

❖ 공백 이중 연결 리스트 생성

```
linkedList_h* createLinkedList_h(void)
{
    linkedList_h* DL;
    // 헤드 노드 할당
    DL = (linkedList_h*)malloc(sizeof(linkedList_h));
    DL->head = NULL;    // 공백 리스트이므로 NULL로 설정
    return DL;
}
```

❖ 이중 연결 리스트를 순서대로 출력

```
void printList(linkedList_h* DL)
{
    listNode* p;
    printf(" DL = (");
    p = DL->head;
    while (p != NULL)
    {
        printf("%s", p->data);
        p = p->rlink;
        if (p != NULL)
            printf(", ");
    }
    printf(") \n");
}
```


❖ 리스트에서 x 노드를 탐색하는 연산

```
listNode* searchNode(linkedList_h* DL, char* x)
{
    listNode *temp;
    temp = DL->head;
    while (temp != NULL)
    {
        if (strcmp(temp->data, x) == 0)
            return temp;
        else
            temp = temp->rlink;
    }
    return temp;
}
```

❖ 프로그램 동작을 위한 메인 함수 만들기

```
int main()
{
    linkedList_h* DL;
    listNode *p;
    DL = createLinkedList_h();      // 공백 리스트 생성
    printf("(1) 이중 연결 리스트 생성하기! \n");
    printList(DL);
    getchar();

    printf("(2) 이중 연결 리스트에 [월] 노드 삽입하기! \n");
    insertNode(DL, NULL, "월");
    printList(DL);
    getchar();

    printf("(3) 이중 연결 리스트의 [월] 노드 뒤에 [수] 노드 삽입하기! \n");
    p = searchNode(DL, "월");
    insertNode(DL, p, "수");
    printList(DL);
    getchar();
}
```

❖ 프로그램 동작을 위한 메인 함수 만들기

```
printf("(4) 이중 연결 리스트의 [수] 노드 뒤에 [금] 노드 삽입하기! \n");  
p = searchNode(DL, "수");  
insertNode(DL, p, "금");  
printList(DL);  
getchar();  
  
printf("(5) 이중 연결 리스트에서 [수] 노드 삭제하기! \n");  
p = searchNode(DL, "수");  
deleteNode(DL, p);  
printList(DL);  
getchar();  
  
return 0;  
}
```

❖ 이중 연결 리스트 프로그램 동작 확인 : [교재 203p](#)

■ 실행 결과

```
C:\₩한국산업기술대학₩2021년1학기₩메카트로닉스₩실습예제₩이중연결리스...
(1) 이중 연결 리스트 생성하기!
DL = ()

(2) 이중 연결 리스트에 [월] 노드 삽입하기!
DL = (월)

(3) 이중 연결 리스트의 [월] 노드 뒤에 [수] 노드 삽입하기!
DL = (월, 수)

(4) 이중 연결 리스트의 [수] 노드 뒤에 [금] 노드 삽입하기!
DL = (월, 수, 금)

(5) 이중 연결 리스트에서 [수] 노드 삭제하기!
DL = (월, 금)

-----
Process exited after 16.68 seconds with return value 0
계속하려면 아무 키나 누르십시오 . . .
```

❖ 원형 연결 리스트를 사용하여 원판 돌리기 게임 만들기

- 참가 인원수 만큼 노드를 만들고 난수를 이용 광과 상품의 존재 여부를 노드에 기록한다.
- 참가 인원을 설정하고 스페이스 키를 눌러 게임을 시작한다.
- 상품이 당첨되지 않으면 메시지를 출력하고 원판 돌리기를 시작한다.
- 만약 상품이 당첨 되면 메시지를 출력하고 종료한다.

❖ 원형 연결 리스트의 노드 구조를 만든다.

```
#include "stdio.h"
#include "stdlib.h"
#include "time.h"
#include "conio.h"

typedef struct ListNode {
    int goods;                // 상품(1: 당첨, 0 : 꽂)
    struct ListNode *link;
} listNode;

typedef struct {
    listNode* head;
} linkedList_h;

void Game();
void MakeSpinWheel(linkedList_h* CL, int n);
void ClearSpinWheel(linkedList_h*);
```

❖ 게임 구성하기

```
void Game()
{
    int n;
    int choice;
    linkedList_h* CL;
    // 헤드노드 할당
    CL = (linkedList_h*)malloc(sizeof(linkedList_h));
    CL->head = NULL;
    printf("참가 인원: ");
    scanf_s("%d", &n);           //게임 참가 인원 입력
    MakeSpinWheel(CL, n);       //참가 인원만큼의 원판 생성
    while (1)
    {
        printf("원판돌리기 시작? 시작: 스페이스키 입력\n");
        choice = _getch();       // 게임 시작 여부 확인
    }
}
```

❖ 게임 구성하기

```
if (choice == 32) {                                // 스페이스 키 확인
    if(CL->head->goods) {                            // 상품이 있으면 ?
        printf("상품 당첨~\n");
        printf("Game Over!!!\n");
        printf("\n");
        ClearSpinWheel(CL);                        // 원판 삭제
        return;                                    // 게임 종료
    }
    else {                                          // 상품이 없으면
        printf("꽁~ 다시한번\n");
        printf("\n");
        CL->head = CL->head->link; //다음으로 이동
    }
}
else {                                            // 스페이스 키를 누리지 않으면
    CL->head = CL->head->link; //다음으로 이동
}
}
```


❖ 원판의 노드 만들기

```
void MakeNode(linkedList_h* CL, int goods)
{
    ListNode *now = (ListNode *)malloc(sizeof(ListNode));
    now->goods = goods;
    if ((CL->head) == NULL)    //빈 상태
    {
        (CL->head) = now;      //맨 앞이 now
        now->link = now;       //현재 now 하나여서 now->next도 now
    }
    else                      //비어 있지 않을 때
    {
        //(*head)와 (*head) 다음 노드 사이에 now를 연결
        now->link = CL->head->link;
        CL->head->link = now;
    }
}
```

❖ 원판 만들기

```
void MakeSpinWheel(linkedList_h* CL, int n)
{
    //value에 1~n 사이의 랜덤 수 발생
    int value = rand() % n + 1;
    CL->head = NULL;           // 리스트의 맨 앞은 NULL로 초기화
    while (n>0)                // 만들 개수가 0보다 클 때
    {
        if (value == n)        // value와 n이 같으면
        {
            MakeNode(CL, 1);    // 상품이 있는 노드 생성
        }
        else                   // 아니면
        {
            MakeNode(CL, 0);    // 상품이 없는 노드 추가
        }
        n--;
    }
}
```

❖ 원판의 노드 제거하기

```
void ClearSpinWheel(linkedList_h* CL)
{
    listNode *seek = CL->head->link; //(*head)의 다음 노드로 초기화
    listNode *prev = CL->head;      // seek의 앞 노드를 (*head)로 설정
    while (seek != CL->head)        // seek가 (*head)가 아니면 반복
    {
        prev = seek;                //seek를 이동하기 전에 prev에 설정
        seek = seek->link;           //seek를 다음 노드로 이동
        free(prev);                  //설정했던 seek의 앞 노드를 제거
    }
    free(CL->head);                  //마지막으로 (*head)노드를 제거
}
```

❖ 게임 실행을 위한 메인 함수 만들기

```
int main()
{
    int key;
    srand((unsigned)time(0)); // 랜덤에 사용할 seed값 설정
    while (1)
    {
        printf("===== 원판 돌리기 게임 =====\n");
        printf("시작하려면 스페이스 키를 누르세요\n");
        printf("종료 하려면 ESC 키를 누르세요\n");
        printf("=====================================\n");
        key = _getch(); // 게임 진행 여부 확인
        if (key == 27) { break; } // 게임 종료 ESC 키 확인
        else if (key == 32) { Game(); } // 게임 시작
        else
            printf("키를 잘못 선택 하셨습니다\n");
        printf("\n");
    }
    return 0;
}
```